

GenAI. Lecture 4 - VAE, VQVAE

A. Ryzhikov¹, S. Ali¹, A. Akhmatova¹, T. Ramazyan¹, K. Aithadzhaev¹, A. Khizhik¹

¹National Research University Higher School of Economics



February 2, 2025



LAMBDA • HSE

Recap



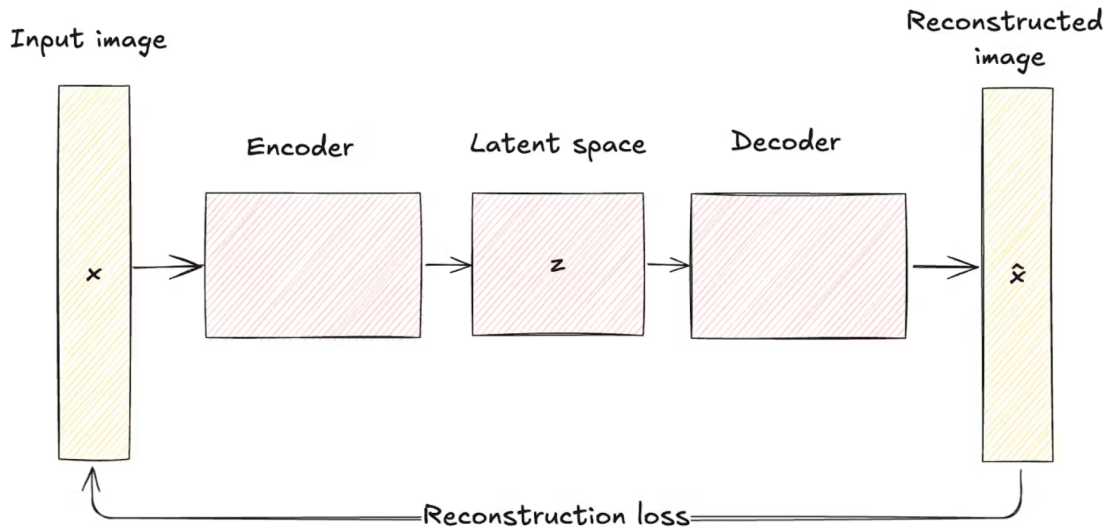
Recap. Key points

- ▶ GAN is unstable, but really fast in practice
- ▶ Wasserstein distance helps to overcome GANs gradient vanishing
- ▶ Several ways to satisfy Lipschitz constraints

Background



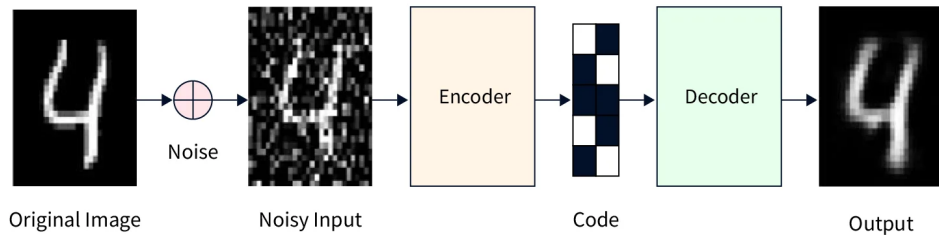
Autoencoder



Autoencoder problems

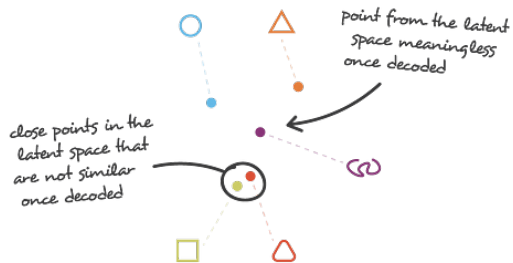
- ▶ overfitting. Irregularity of latent space
- ▶ $p(z)$ is unknown

Denoising Autoencoder

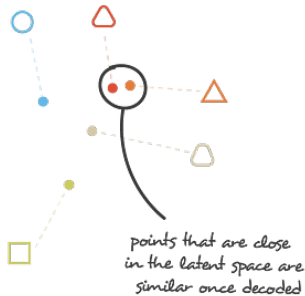


SCALER
Topics

Irregularity of latent space



irregular latent space



regular latent space



Variational Autoencoder (VAE)



Log likelihood

Let's maximize $\log p(x)$

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)}$$

$$\log p(x) = \log \frac{p(x|z)p(z)}{p(z|x)}$$

$$\log p(x) = \int q(z) \log \frac{p(x|z)p(z)}{p(z|x)} dz.$$

$$\log p(x) = \int q(z) \log \frac{p(x|z)p(z)q(z)}{p(z|x)q(z)} dz.$$

$$\log p(x) = \mathbb{E}_{q(z)}[\log p(x|z)] + D_{\text{KL}}(q(z)||p(z|x)) - D_{\text{KL}}(q(z)||p(z)).$$

ELBO

$$\log p(x) = \mathbb{E}_{q(z)}[\log p(x|z)] - D_{\text{KL}}(q(z)\|p(z)) + D_{\text{KL}}(q(z)\|p(z|x)) \geq$$

$$[\mathbb{E}_{q(z)}[\log p(x|z)] - D_{\text{KL}}(q(z)\|p(z))] = L(q(z))$$

$L(q(z))$ is **Evidence Lower BOund (ELBO)**

KL with prior

Let's assume $p(z) = \mathcal{N}(0, 1)$, $p(z|x) = \mathcal{N}(\mu_x, \sigma_x)$, $q(z|x) = \mathcal{N}(\mu(x), \sigma(x))$

- Measures the degree to which the auxiliary distribution, $q(z, \eta)$, matches the prior $p(z)$:

$$KL(q(z; \eta) \| p(z)).$$

- The KL divergence has a closed form when both distributions are parametric (e.g., Gaussian):

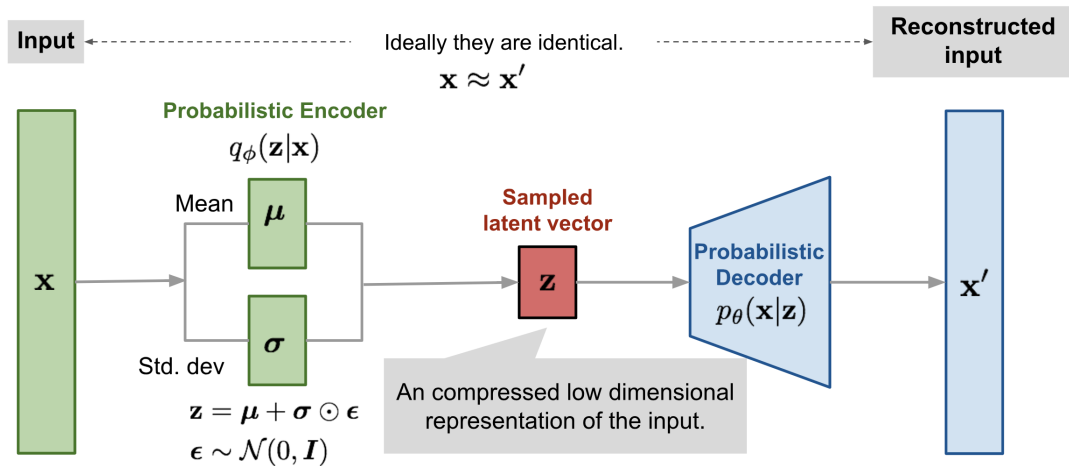
$$q(z; \eta) = \mathcal{N}_z(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

$$p(z) = \mathcal{N}_z(\mathbf{0}, \mathbf{I}).$$

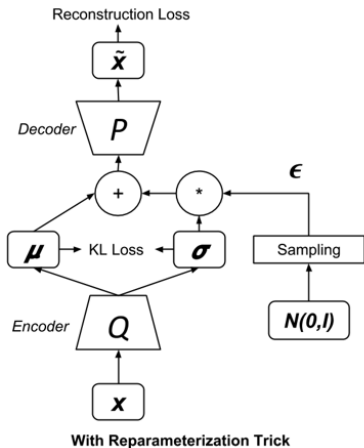
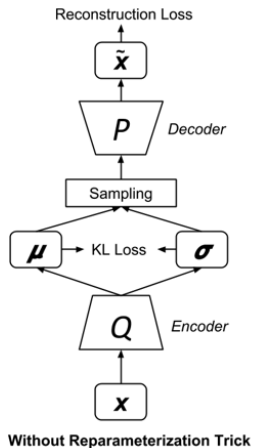
- In this case:

$$KL[q(z|x, \eta), p(z)] = \frac{1}{2} \left(\text{Tr}[\boldsymbol{\Sigma}] + \boldsymbol{\mu}^T \boldsymbol{\mu} - D - \log \det[\boldsymbol{\Sigma}] \right).$$

Architecture



Reparametrization Trick



Training algorithm

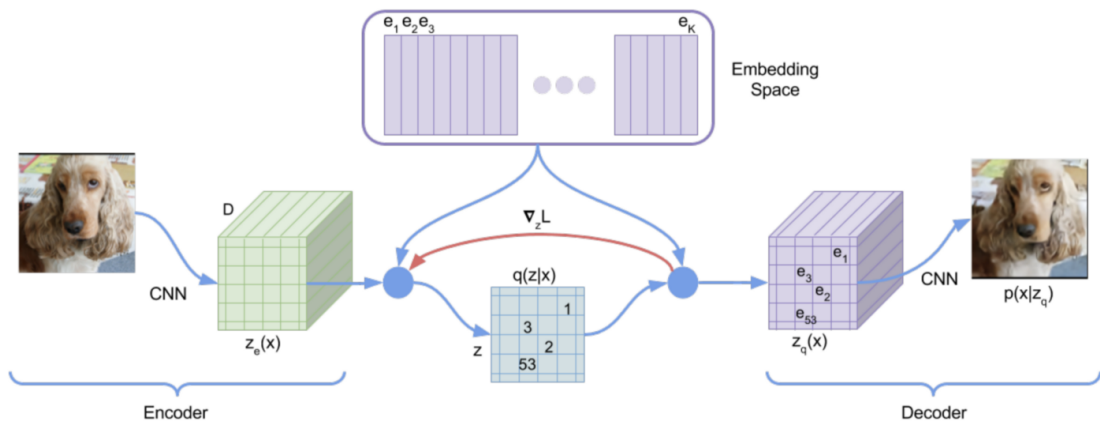
```
1: Input: Dataset  $X = \{x_1, x_2, \dots, x_N\}$ , Learning rate  $\alpha$ , Batch size  $B$ 
2: Initialize: Encoder parameters  $\phi$  and decoder parameters  $\theta$ 
3: for epoch in range(num_epochs) do
4:   for minibatch  $\{x_1, \dots, x_B\}$  in dataset do
5:     Encode: Compute  $\mu, \log \sigma^2 = \text{Encoder}(x)$ 
6:     Compute  $\sigma = \exp(0.5 \cdot \log \sigma^2)$ 
7:     Sample  $\epsilon \sim \mathcal{N}(0, I)$ 
8:     Compute latent variable  $z = \mu + \sigma \cdot \epsilon$ 
9:     Decode: Compute reconstructed  $x_{\text{recon}} = \text{Decoder}(z)$ 
10:    Compute Reconstruction Loss:
11:    recon_loss =  $-\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] = \text{MSE}(x, x_{\text{recon}})$ 
12:    Compute KL Divergence Loss:
13:    KL_loss =  $\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \exp(\log \sigma^2))$ 
14:    loss = recon_loss + KL_loss
15:    Optimizer.zero_grad()
16:    loss.backward()
17:    Optimizer.step()
18:   end for
19: end for
20: Output: Trained decoder  $v_\theta(x|z)$ 
```

▷ Reparameterization trick

VQ-VAE



VQ-VAE



Algorithm pseudocode

```
1: Input: Dataset  $X = \{x_1, x_2, \dots, x_N\}$ , Codebook size  $K$ , Learning rate  $\alpha$ , Batch size  $B$ 
2: Initialize: Encoder parameters  $\phi$ , decoder parameters  $\theta$ , codebook  $E = \{e_1, \dots, e_K\}$ 
3: for epoch in range(num_epochs) do
4:   for minibatch  $\{x_1, \dots, x_B\}$  in dataset do
5:      $z_e = \text{Encoder}_\phi(x)$  ▷ Encode input into continuous latent representation
6:      $z_q = \arg \min_{e_k} \|z_e - e_k\|_2$  ▷ Quantization: Assign  $z_e$  to nearest codebook entry
7:      $x_{\text{recon}} = \text{Decoder}_\theta(z_q)$  ▷ Decode quantized latent representation
8:      $L_{\text{recon}} = -\mathbb{E} \log p_\theta(x|z_q)$  ▷ Compute Reconstruction Loss
9:      $L_{\text{vq}} = \|z_e - \text{sg}(z_q)\|_2^2$  ▷ Compute Codebook Loss (Vector Quantization Error)
10:     $L_{\text{commit}} = \beta \|\text{sg}(z_e) - z_q\|_2^2$  ▷ Compute Commitment Loss
11:     $L = L_{\text{recon}} + L_{\text{vq}} + L_{\text{commit}}$  ▷ Total Loss: Combining All Terms
12:    Optimizer.zero_grad() ▷ Backpropagation and Optimization
13:    Loss.backward()
14:    Optimizer.step()
15:   end for
16: end for
17: Output: Trained discrete decoder  $p_\theta(z_q)$ 
```

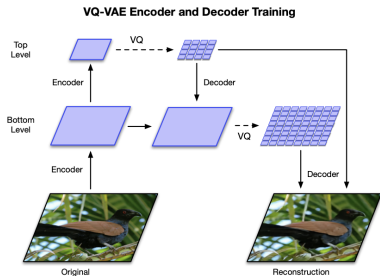
VQ-VAE-2: Key Differences from VQ-VAE

- ▶ **Hierarchical Latent Space:** Uses multiple latent levels (coarse-to-fine representation).
- ▶ **EMA Codebook Updates:** Replaces gradient-based learning with an Exponential Moving Average (EMA).
- ▶ **Improved Training Stability:** Reduces the risk of posterior collapse.
- ▶ **Better Image Quality:** Enables high-resolution image generation (e.g., DALL-E).
- ▶ Learned (PixelCNN) prior

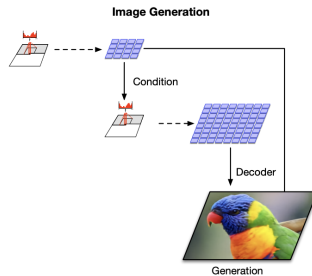
Hierarchical Latent Representation

- ▶ VQ-VAE-2 introduces a three-level hierarchy:
 - **Top Level:** Captures global structure.
 - **Middle Level:** Adds more details.
 - **Bottom Level:** Preserves fine-grained textures.
- ▶ This structure helps in generating detailed, high-resolution images.

VQ-VAE 2



(a) Overview of the architecture of our hierarchical VQ-VAE. The encoders and decoders consist of deep neural networks. The input to the model is a 256×256 image that is compressed to quantized latent maps of size 64×64 and 32×32 for the *bottom* and *top* levels, respectively. The decoder reconstructs the image from the two latent maps.



(b) Multi-stage image generation. The top-level PixelCNN prior is conditioned on the class label, the bottom level PixelCNN is conditioned on the class label as well as the first level code. Thanks to the feed-forward decoder, the mapping between latents to pixels is fast. (The example image with a parrot is generated with this model).

EMA Codebook Updates

- ▶ Codebook entries are updated via Exponential Moving Average (EMA):

$$e_k \leftarrow \gamma e_k + (1 - \gamma) \cdot \frac{\sum_i m_i z_e^i}{\sum_i m_i}$$

- ▶ This prevents posterior collapse and improves stability.

VQ-VAE 2. Pseudocodealgorithm

Algorithm 1 VQ-VAE training (stage 1)

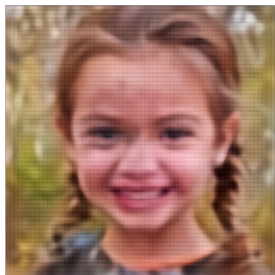
Require: Functions E_{top} , E_{bottom} , D , $\mathbf{x}$
(batch of training images)

- 1: $\mathbf{h}_{top} \leftarrow E_{top}(\mathbf{x})$
▷ quantize with top codebook eq 1
 - 2: $\mathbf{e}_{top} \leftarrow \text{Quantize}(\mathbf{h}_{top})$
 - 3: $\mathbf{h}_{bottom} \leftarrow E_{bottom}(\mathbf{x}, \mathbf{e}_{top})$
▷ quantize with bottom codebook eq 1
 - 4: $\mathbf{e}_{bottom} \leftarrow \text{Quantize}(\mathbf{h}_{bottom})$
 - 5: $\hat{\mathbf{x}} \leftarrow D(\mathbf{e}_{top}, \mathbf{e}_{bottom})$
▷ Loss according to eq 2
 - 6: $\theta \leftarrow \text{Update}(\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}))$
-

Algorithm 2 Prior training (stage 2)

- 1: $\mathbf{T}_{top}, \mathbf{T}_{bottom} \leftarrow \emptyset$ ▷ training set
 - 2: **for** $\mathbf{x} \in \text{training set}$ **do**
 - 3: $\mathbf{e}_{top} \leftarrow \text{Quantize}(E_{top}(\mathbf{x}))$
 - 4: $\mathbf{e}_{bottom} \leftarrow \text{Quantize}(E_{bottom}(\mathbf{x}, \mathbf{e}_{top}))$
 - 5: $\mathbf{T}_{top} \leftarrow \mathbf{T}_{top} \cup \mathbf{e}_{top}$
 - 6: $\mathbf{T}_{bottom} \leftarrow \mathbf{T}_{bottom} \cup \mathbf{e}_{bottom}$
 - 7: **end for**
 - 8: $p_{top} = \text{TrainPixelCNN}(\mathbf{T}_{top})$
 - 9: $p_{bottom} = \text{TrainCondPixelCNN}(\mathbf{T}_{bottom}, \mathbf{T}_{top})$
▷ Sampling procedure
 - 10: **while** true **do**
 - 11: $\mathbf{e}_{top} \sim p_{top}$
 - 12: $\mathbf{e}_{bottom} \sim p_{bottom}(\mathbf{e}_{top})$
 - 13: $\mathbf{x} \leftarrow D(\mathbf{e}_{top}, \mathbf{e}_{bottom})$
 - 14: **end while**
-

VQ-VAE 2



h_{top}



$h_{\text{top}}, h_{\text{middle}}$



$h_{\text{top}}, h_{\text{middle}}, h_{\text{bottom}}$



Original

Applications of VQ-VAE-2

- ▶ **DALL-E:** Enables high-resolution text-to-image generation.
- ▶ **Speech Audio Modeling:** Used for realistic speech synthesis.
- ▶ **Video Generation:** Helps generate smooth and consistent videos.
- ▶ **Language Modeling:** Provides structured latent representations for NLP models.

Summary of VQ-VAE-2

- ▶ Introduces a **hierarchical latent space** for improved image quality.
- ▶ Uses **EMA-based codebook updates** for better training stability.
- ▶ Reduces **posterior collapse** compared to VQ-VAE.
- ▶ Enables **high-resolution generative models** like DALL-E.

Conclusion



Conclusion

- ▶ Autoencoders map data to a lower-dimensional space but suffer from irregular latent spaces and unknown priors.
- ▶ Denoising Autoencoders improve robustness by reconstructing noisy inputs.
- ▶ Variational Autoencoders (VAEs) introduce a probabilistic latent space, optimizing the **ELBO** objective.
- ▶ Reparameterization trick allows backpropagation through stochastic latent variables.
- ▶ VAEs minimize **KL divergence** between the approximate and true posterior.
- ▶ VQ-VAE replaces continuous latent variables with discrete codebook vectors, enabling more structured representation.
- ▶ Vector quantization enforces latent space regularity and prevents posterior collapse.
- ▶ VQ-VAE-2 introduces a **hierarchical multi-scale** latent space for improved high-resolution generation.
- ▶ Codebook updates in VQ-VAE-2 use **Exponential Moving Average (EMA)** for stability.
- ▶ PixelCNN priors model dependencies in latent space, improving generative performance.
- ▶ VQ-VAE-2 enables applications in **high-resolution image synthesis, speech generation, and video modeling**.